Reviewer Pack — Sum-Product Complexity Lower Bounds for ACC⁰ Circuits

Entry 938bcc7ab3fd · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-13 04:27:28 UTC

Sum-Product Complexity Lower Bounds for ACC⁰ Circuits

Entry ID: `938bcc7ab3fd` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-13 04:27:28 UTC

1. Conjecture

Field A (mathematical branch): Additive Combinatorics **Field B** (complexity object): ACC⁰ Circuit Size for Sipser-like Functions

Statement:

> For any explicit function f: {0,1}^n → {0,1} with sum-product complexity Ω(n²), the minimal ACC⁰ circuit size computing f is Ω(2^{n/2}).

Rationale (proposer's reasoning):

> Sum-product growth captures structural complexity that may resist ACC⁰ computation, as suggested by the polynomial method's failure to resolve ACC⁰ lower bounds. This links additive energy to circuit size via explicit function constraints.

Taxonomy category: `ACC_SIPSER` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2def30894730d3ba`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        pivot = A[i][i]
        if pivot == 0:
            return None
        for j in range(i + 1, m):
            factor = -A[j][i] / pivot
            for k in range(n):
                A[j][k] += factor * A[i][k]
    return A

def matrix_multiplication(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def sipser_like_function(n, seed):
        random.seed(seed)
        return [random.randint(0, 1) for _ in range(n)]

    def sum_product_complexity(A):
        A_plus = set()
        A_dot = set()
        for x in A:
            if x not in A_plus:
                A_plus.add(x)
            if x not in A_dot:
                A_dot.add(tuple(sorted(x)))
        return len(A_plus) * len(A_dot)

    def dpll_circuit(f, n):
        def evaluate(circuit, assignment):
            for gate in circuit:
                if gate['type'] == 'AND':
```

```
                    result = True
                    for input in gate['inputs']:
                        if not evaluate(input, assignment):
                            result = False
                            break
                    assignment[gate['output']] = result
                elif gate['type'] == 'OR':
                    result = False
                    for input in gate['inputs']:
                        if evaluate(input, assignment):
                            result = True
                            break
                    assignment[gate['output']] = result
                elif gate['type'] == 'NOT':
                    assignment[gate['output']] = not evaluate(gate['input'], assignment)
            return assignment[f]

        def backtrack(circuit, assignment):
            if all(assignment[x] is not None for x in f):
                return evaluate(circuit, assignment)
            var = next(x for x in f if assignment[x] is None)
            for val in [True, False]:
                assignment[var] = val
                if backtrack(circuit, assignment):
                    return True
                assignment[var] = None
            return False

        circuit = []
        for i in range(2**n):
            inputs = [(i >> j) & 1 for j in range(n)]
            output = f[i]
            if output == 1:
                circuit.append({'type': 'AND', 'inputs': [{'type': 'NOT' if x else 'VAR', 'input':
        return backtrack(circuit, {x: None for x in f})

    n = random.choice([5, 10, 15, 20, 30, 40])
    f = sipser_like_function(n, seed)
    A = [i for i, x in enumerate(f) if x == 1]
    sp_complexity = sum_product_complexity(A)

    if sp_complexity < n**2:
        return {
            "metric_name": "sum_product_complexity",
            "metric_value": sp_complexity,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "function_has_lower_sp_complexity"
        }

    acc0_size = 2**(n//2)
    instances_tested = 0
    conjecture_holds = True

    for _ in range(30):
        if dpll_circuit(f, n):
```

```
            instances_tested += 1

    return {
        "metric_name": "sum_product_complexity",
        "metric_value": sp_complexity,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r['metric_value'] * r['instances_tested'] for r in results) / sum(r['instance
    std_value = math.sqrt(sum((r['metric_value'] - mean_value)**2 * r['instances_tested'] for r in
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r['conjecture_holds'] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result['conj
        print(f"RESULT: FALSIFIED counterexample=\"function_has_lower_sp_complexity\" first_failin
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ffa418c4.py", line 132, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ffa418c4.py", line 99, in run_trial
    sp_complexity = sum_product_complexity(A)
                    ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ffa418c4.py", line 53, in sum_product_c
    A_dot.add(tuple(sorted(x)))
                    ^^^^^^^^^
TypeError: 'int' object is not iterable
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed with TypeError due to non-iterable 'int' object in sum_product_complexity function | next: Fix the TypeError in test_ffa418c4.py's sum_product_complexity function by ensuring inputs are iterable

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	36755
2	preregistration	ollama_remote	qwen3:8b	0	0	24155
3	novelty	ollama_remote	qwen3:8b	0	0	21017
4	novelty	ollama_remote	qwen3:8b	0	0	17249
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15196
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13715
7	judge	ollama_remote	qwen3:8b	0	0	19178

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 147266 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/938bcc7ab3fd.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/938bcc7ab3fd.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/938bcc7ab3fd.tar.gz` (if generated)